LAPORAN PRAKTIKUM

POSTTEST 8

ALGORITMA PEMROGRAMAN DASAR

Disusun oleh:

Cici Muzizah Nur Novita Putri (2609106008)

Kelas A1 ‘26

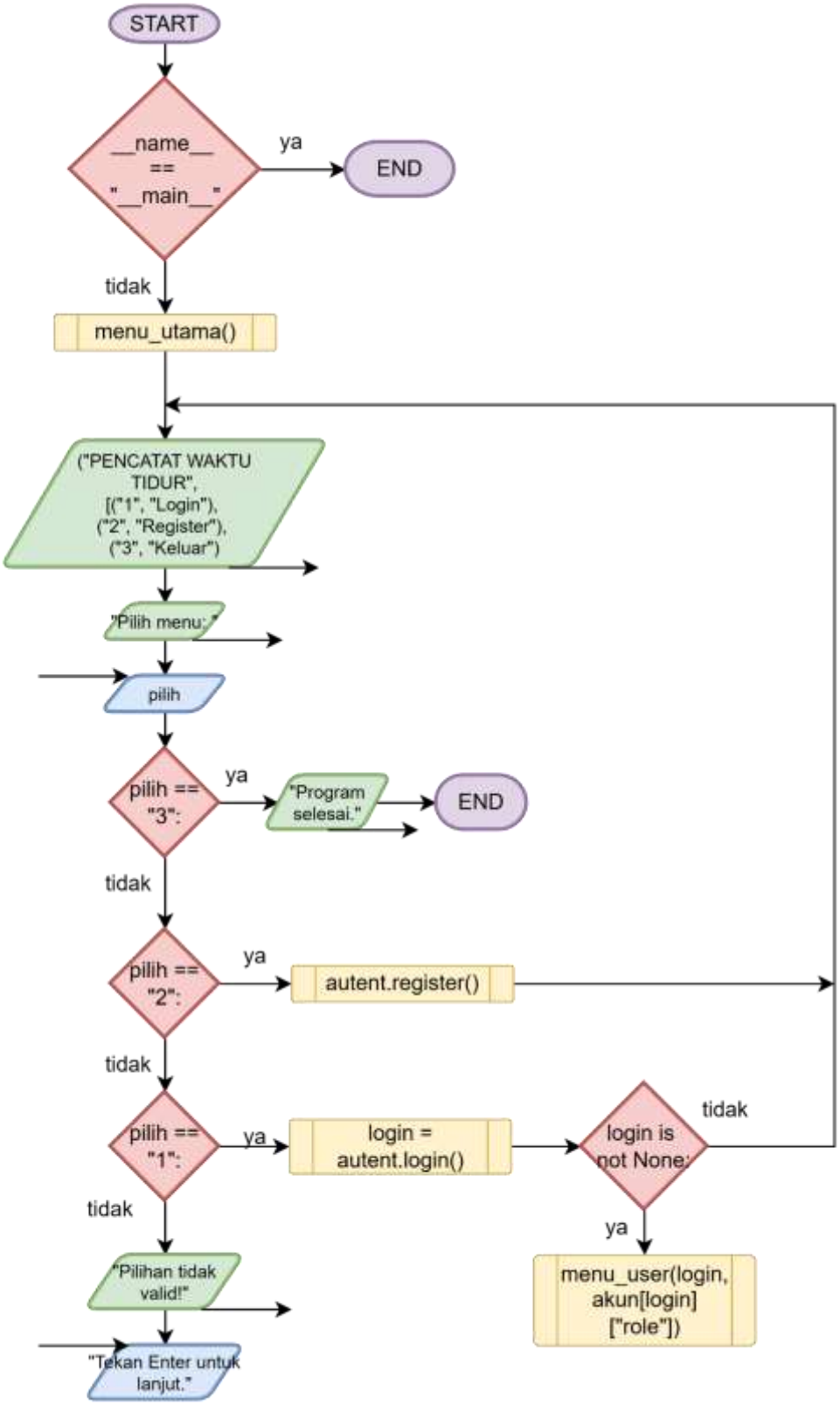
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

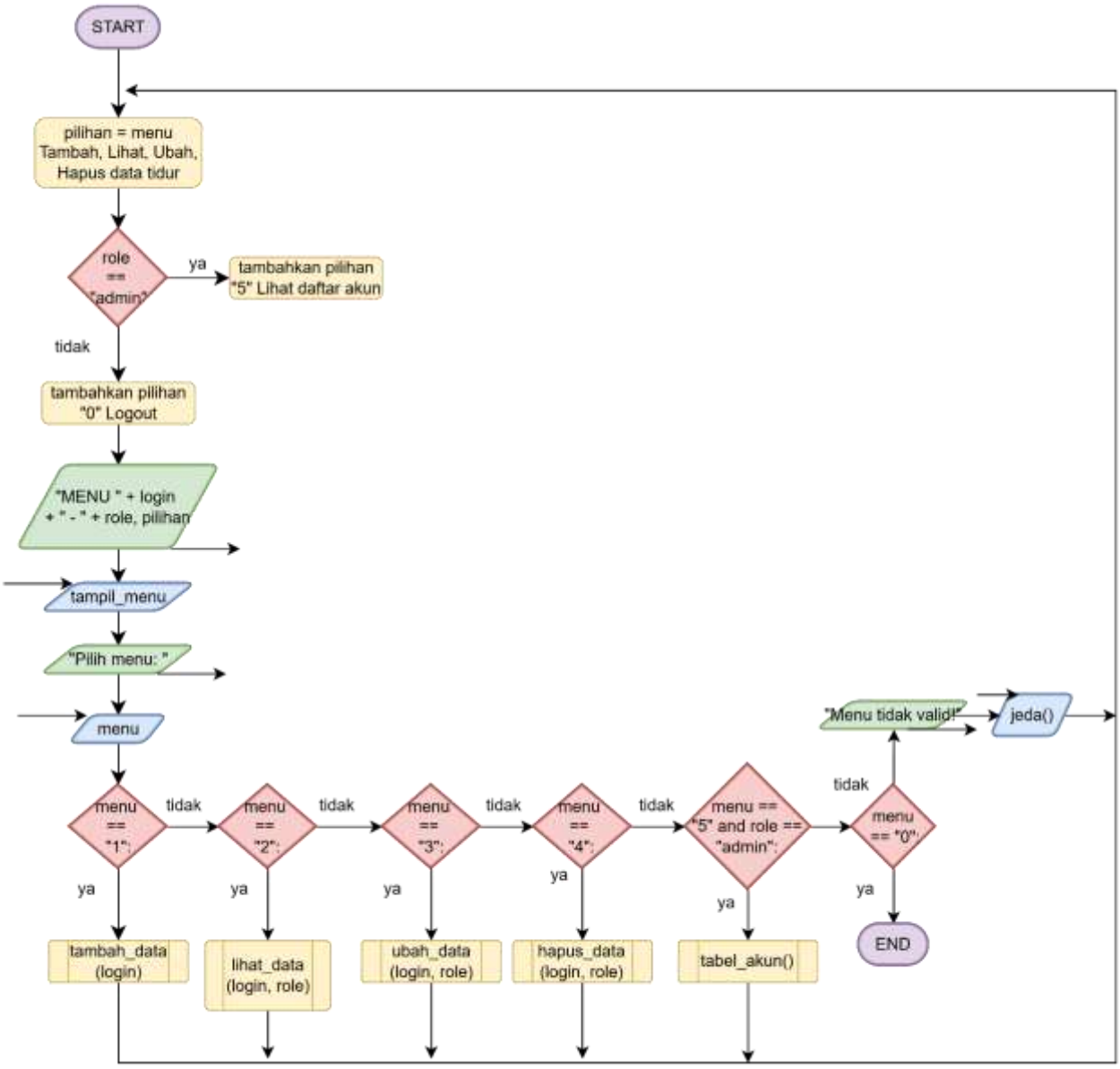
SAMARINDA

2026

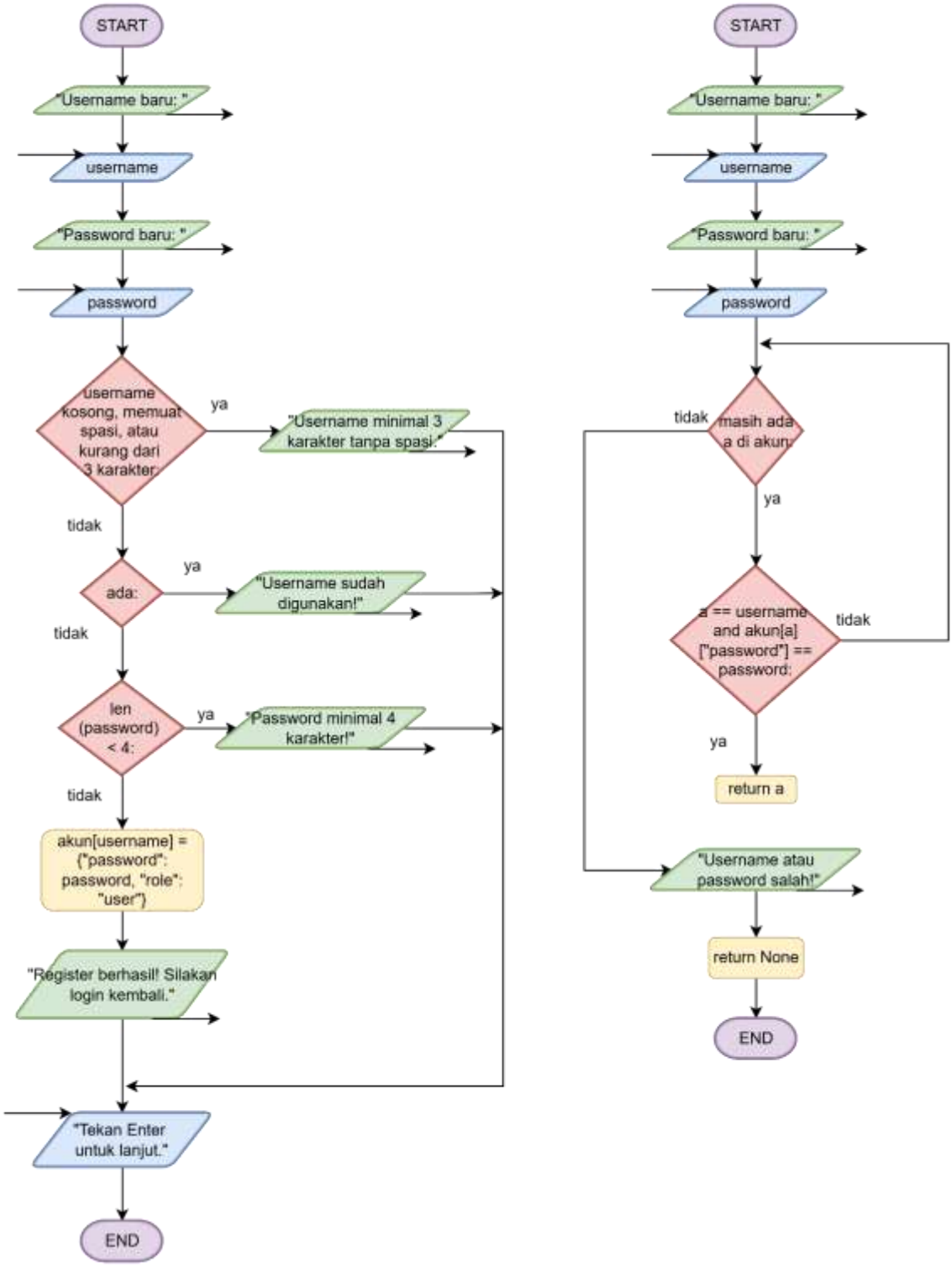
1. Flowchart



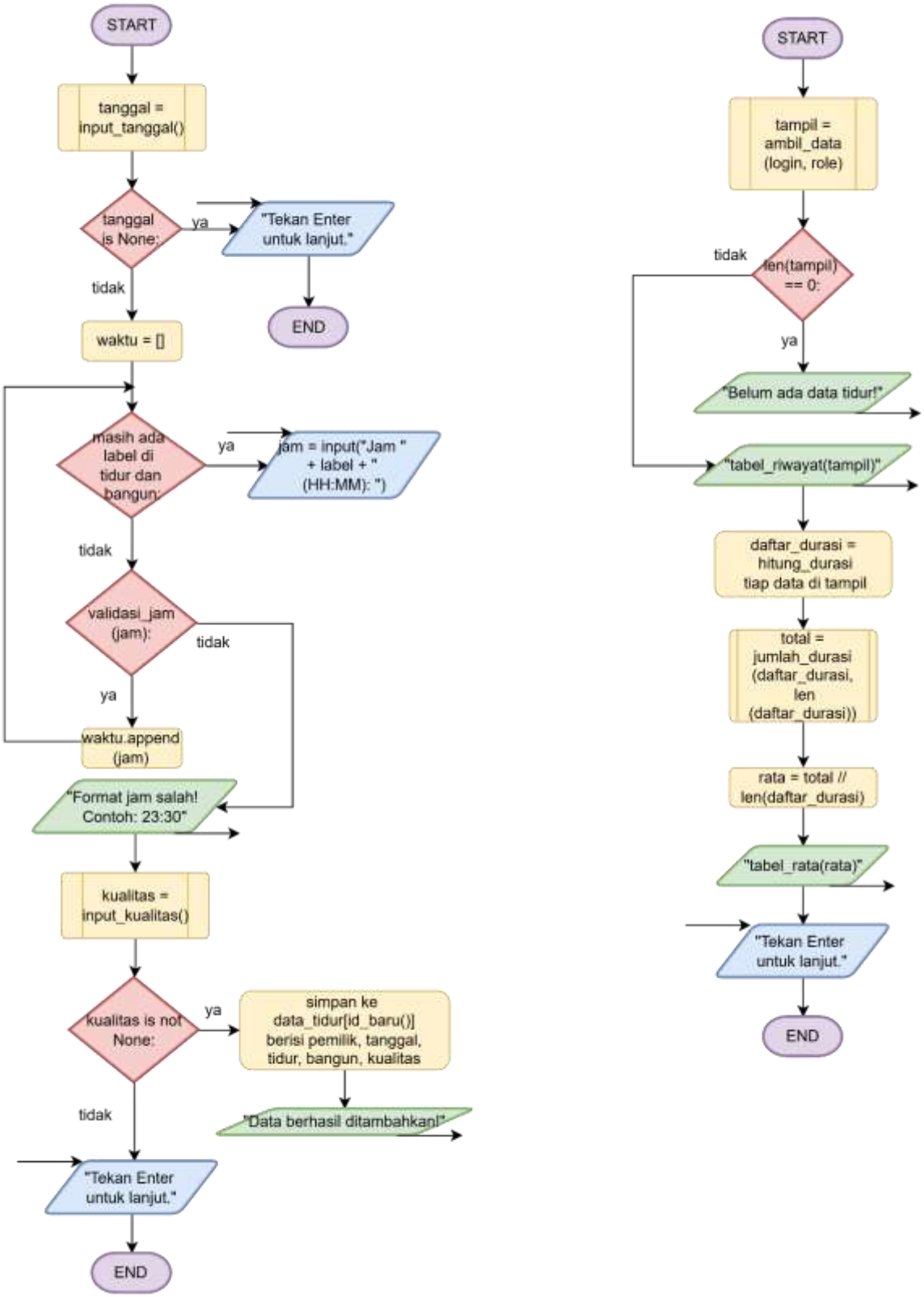
Gambar 1.1 flowchart bagian main.py dan menu_utama()



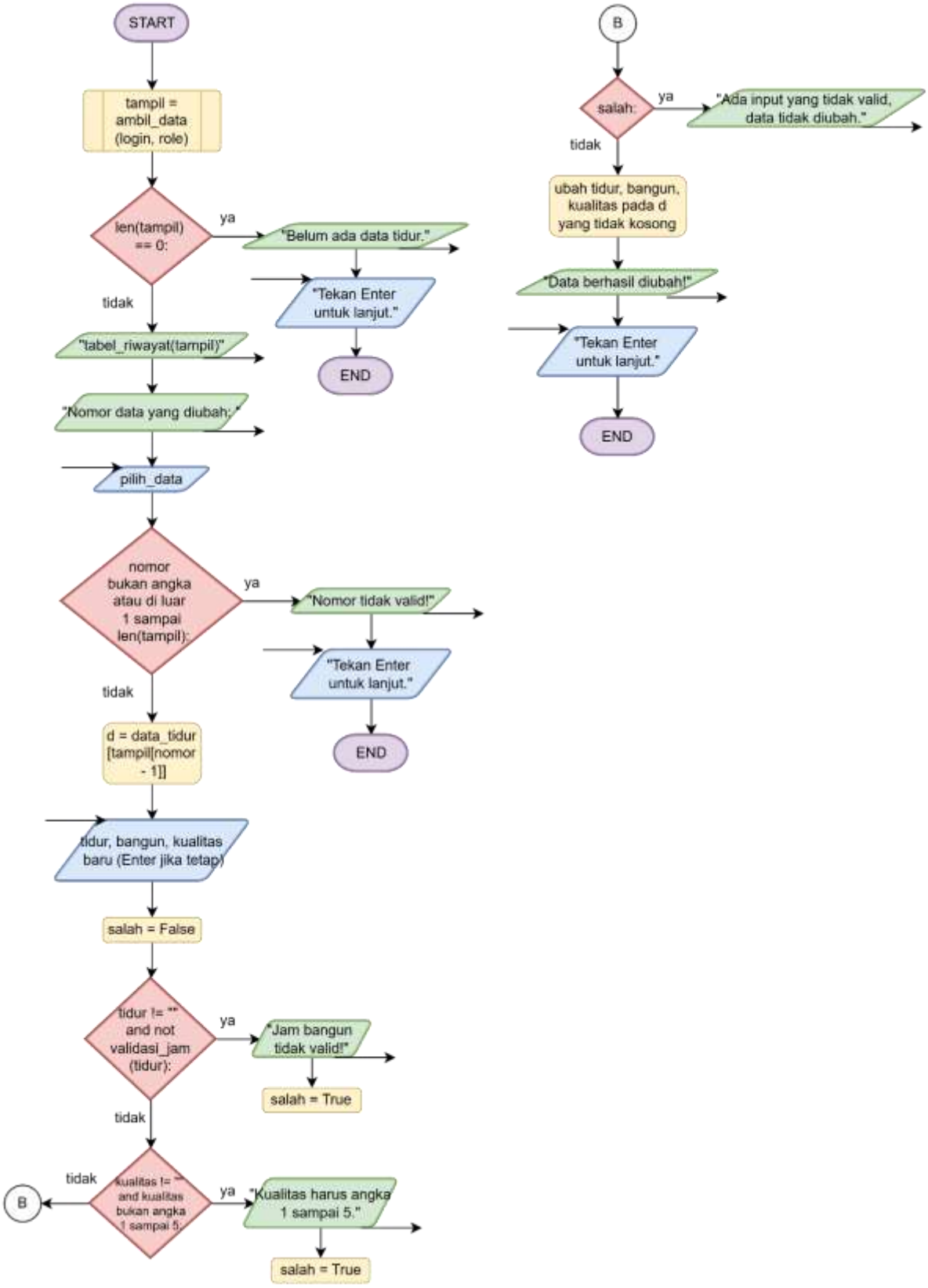
Gambar 1.2 flowchart bagian menu_user(login, role)



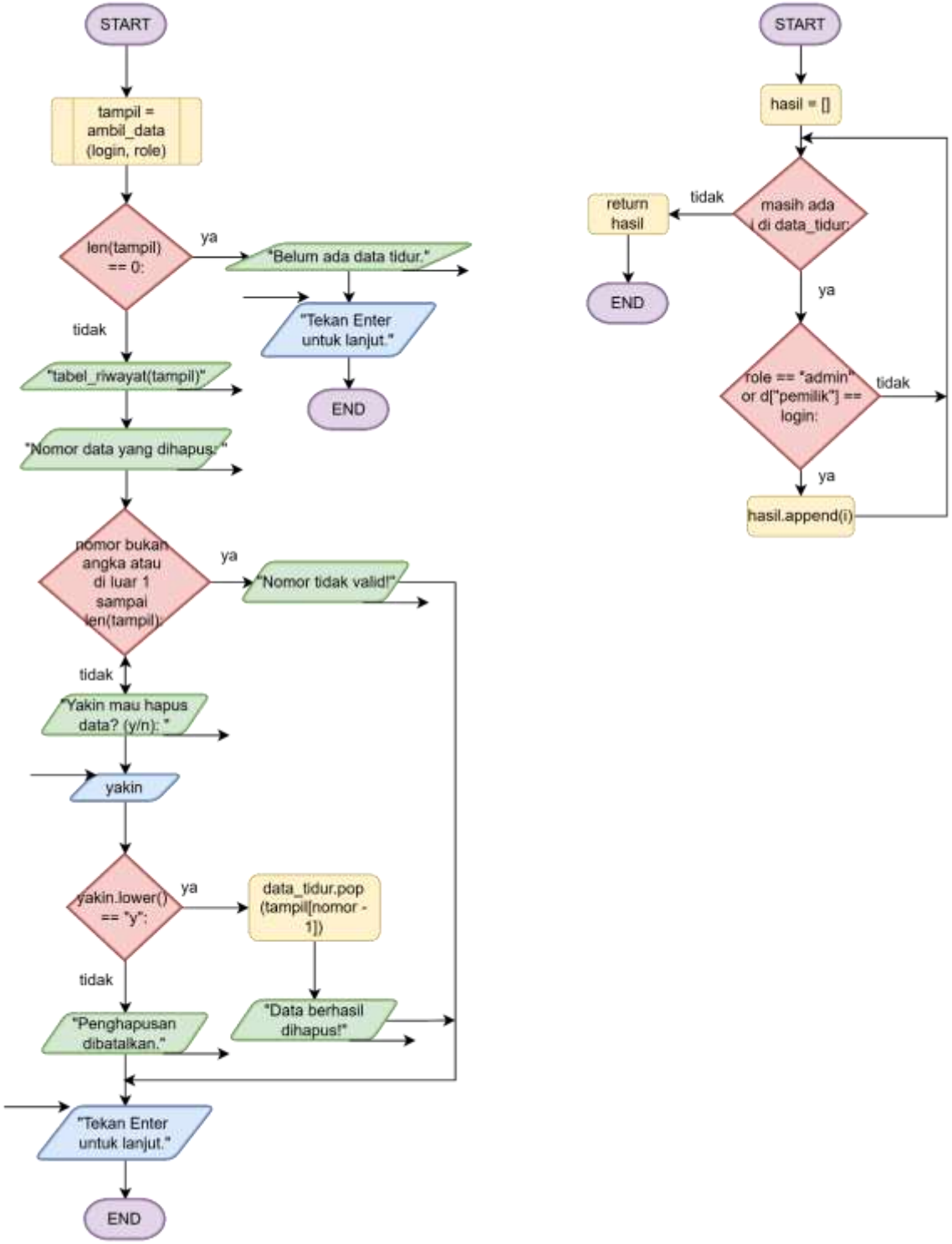
Gambar 1.3 flowchart bagian register() dan login()



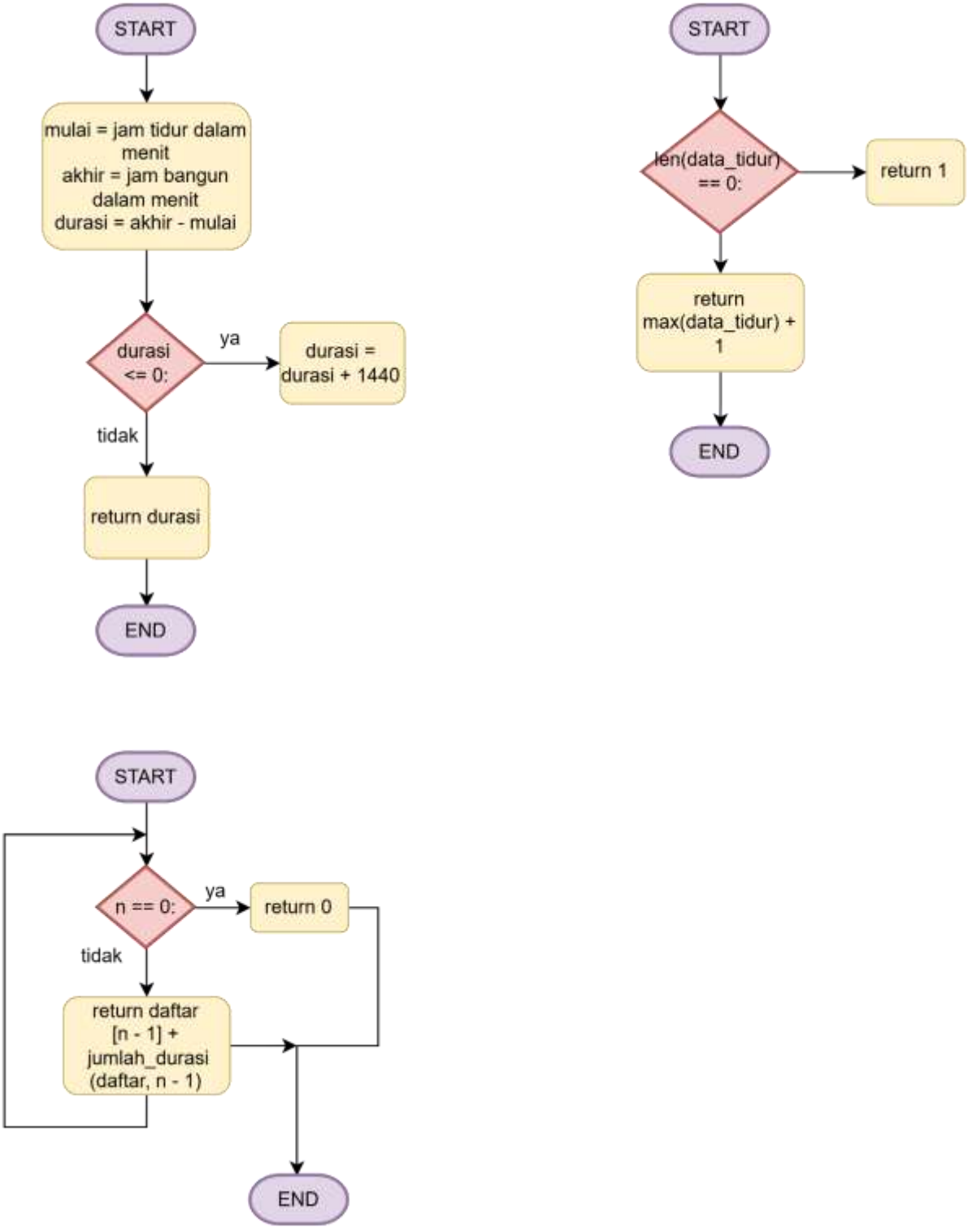
Gambar 1.4 flowchart bagian tambah_data() dan lihat_data()



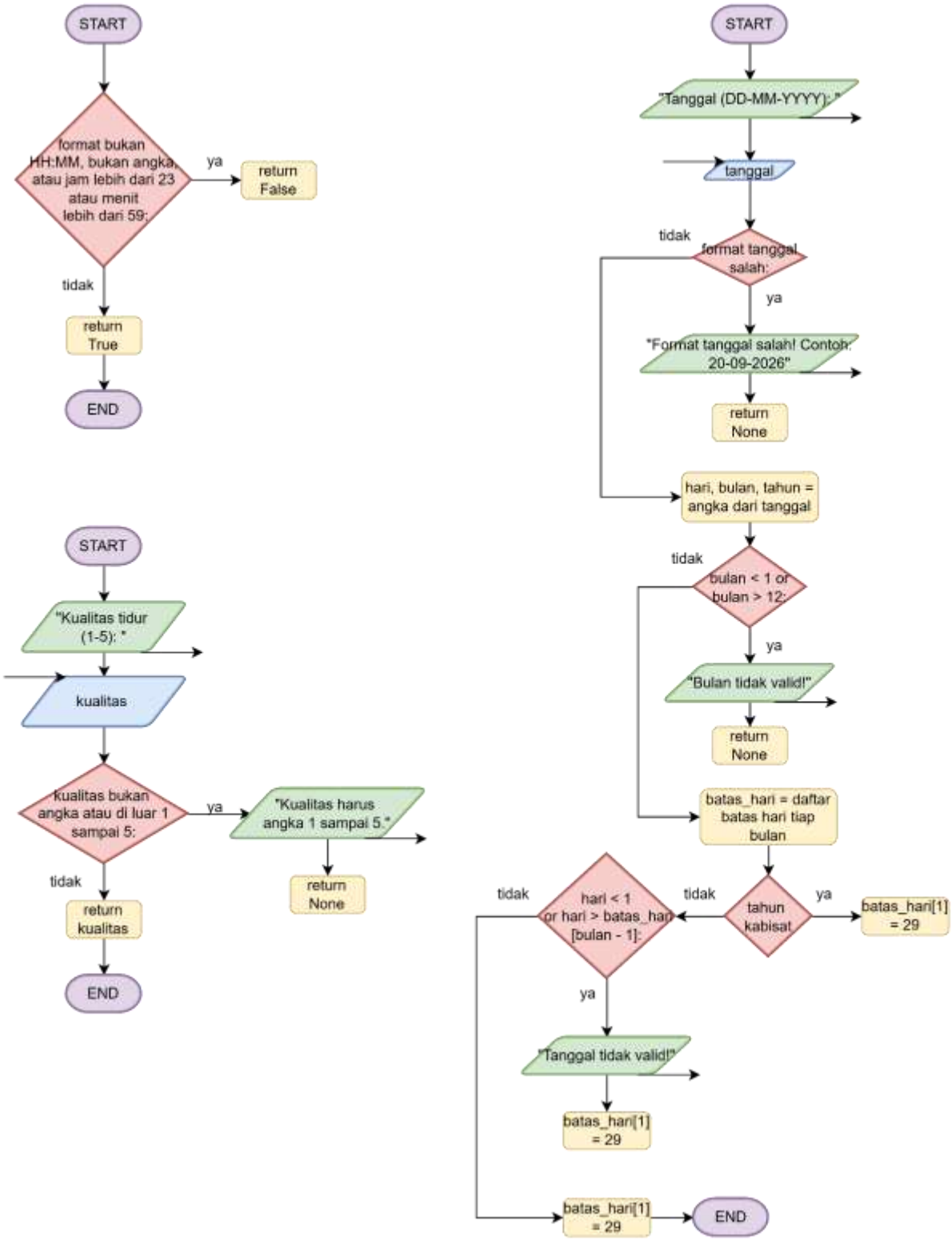
Gambar 1. 5 flowchart bagian ubah_data()



Gambar 1.6 flowchart bagian hapus_data() dan ambil_data()



Gambar 1.7 flowchart bagian hitung_durasi, jumlah_durasi, id_baru



Gambar 1.8 flowchart bagian validasi_jam, input_kualitas, input_tanggal

Flowchart dibagi menjadi delapan halaman, dan setiap fungsi digambar dengan flowchart tersendiri. Flowchart dimulai dari simbol START pada main.py, yang memeriksa apakah file dijalankan langsung, lalu memanggil fungsi menu_utama() dari menu.py. Program dipecah menjadi tujuh file berdasarkan fungsionalitasnya, yaitu data.py yang menyimpan dictionary akun, data_tidur, dan fungsi id_baru(), tampilan.py yang mengatur output PrettyTable, pesan berwarna, serta perhitungan durasi, validasi.py yang memeriksa input jam, tanggal, dan kualitas, autent.py yang menangani register dan login, crud.py yang menangani tambah, lihat, ubah, dan hapus data tidur, menu.py yang mengatur menu, dan main.py sebagai file yang dijalankan. Pemanggilan fungsi digambar dengan simbol predefined process. Pada halaman pertama, program menampilkan menu utama berupa tabel PrettyTable yang terdiri dari Login, Register, dan Keluar. Jika pengguna memilih Keluar dengan angka "3", program menampilkan "Program selesai." lalu diakhiri dengan simbol END. Jika pilihan tidak valid, program menampilkan pesan kesalahan lalu kembali ke awal perulangan menu utama.

Jika pengguna memilih Register, program memanggil register() yang meminta username dan password baru, dengan password disembunyikan menggunakan pwinput. Program memeriksa apakah username sudah ada, lalu memvalidasi input. Jika username kurang dari 3 karakter atau memuat spasi, username sudah dipakai, atau password kurang dari 4 karakter, program menampilkan pesan kesalahan berwarna merah. Pada kode, pemeriksaan ini dijalankan di dalam blok try dengan raise ValueError, dan pesannya ditampilkan oleh blok except. Jika tidak ada error, blok else menyimpan akun baru dengan role "user" dan menampilkan pesan berhasil berwarna hijau, lalu blok finally selalu menampilkan "Tekan Enter untuk lanjut." Jika pengguna memilih Login, program memanggil login() yang mencocokkan username dan password dengan data akun. Jika cocok, fungsi mengembalikan username dan program memanggil menu_user() dengan role dari akun tersebut. Jika tidak cocok, program menampilkan pesan kesalahan dan kembali ke menu utama.

Pada menu_user(), program menampilkan menu pengelolaan data tidur dalam bentuk tabel PrettyTable, yaitu Tambah, Lihat, Ubah, dan Hapus data tidur. Jika role adalah admin, tabel ditambah pilihan nomor 5 untuk melihat daftar username dan role semua akun. Pilihan Logout dengan angka "0" menghentikan perulangan menu sehingga program kembali ke menu utama, sedangkan pilihan yang tidak tersedia menampilkan pesan "Menu tidak valid!" lalu kembali ke awal perulangan menu_user().

Fitur Tambah pada tambah_data() memanggil input_tanggal() untuk memvalidasi tanggal, validasi_jam() untuk jam tidur dan jam bangun, serta input_kualitas() untuk kualitas, sebelum data disimpan dengan ID dari fungsi id_baru(). Fitur Lihat pada lihat_data() mengambil data yang boleh diakses melalui ambil_data(), yaitu data semua pengguna untuk admin dan data milik sendiri untuk user. Data ditampilkan dengan tabel PrettyTable, durasi tiap data dihitung dengan hitung_durasi(), lalu rata-rata durasi dihitung menggunakan fungsi rekursif jumlah_durasi(). Fitur Ubah pada ubah_data() memvalidasi nomor data dan kualitas serta jam dengan validasi_jam(), dan data hanya diubah jika semua input valid. Fitur Hapus pada hapus_data() memvalidasi nomor data lalu meminta konfirmasi "y" atau "n" sebelum data dihapus dari dictionary.

Halaman terakhir berisi fungsi pendukung. Fungsi hitung_durasi() menghitung selisih jam bangun dan jam tidur dalam menit, dan menambahkan 1440 menit jika hasilnya nol atau negatif karena tidur melewati tengah malam. Fungsi jumlah_durasi() menjumlahkan daftar durasi secara rekursif sampai n bernilai 0. Fungsi id_baru() mengembalikan 1 jika data kosong, atau ID terbesar ditambah 1. Fungsi validasi_jam(), input_kualitas(), dan input_tanggal() memeriksa format input, lalu mengembalikan nilai yang valid atau None jika input salah.

2. Deskripsi Singkat Program

Program ini dibuat untuk mencatat dan memantau kebiasaan tidur pengguna secara sederhana. Program mencakup fitur login dan register, pembagian hak akses antara admin dan pengguna biasa, serta operasi CRUD pada data tidur, yaitu menambah, melihat, mengubah, dan menghapus data. Program juga menghitung durasi setiap tidur dan rata-rata durasi tidur agar pengguna dapat menilai pola tidurnya. Program dikembangkan dengan menampilkan output menggunakan library PrettyTable, yaitu pada menu, riwayat tidur, rata-rata durasi, dan daftar akun. Kode juga dipecah menjadi tujuh file berdasarkan fungsionalitasnya (pemrograman modular) agar lebih rapi.

Program memakai library pwinput untuk menyembunyikan password saat diketik dan library colorama untuk memberi warna hijau pada pesan berhasil dan merah pada pesan gagal. Input divalidasi dengan error handling try-except sehingga program tidak berhenti saat input salah. Program ini bermanfaat untuk melatih penggunaan dictionary, perulangan, percabangan, fungsi, prosedur, rekursif, error handling, library eksternal, dan pemrograman modular.

3. Source Code

File data.py

```
akun = {
    "admin": {"password": "admin123", "role": "admin"},
    "xavv": {"password": "xavv201", "role": "user"}
}

data_tidur = {
    1: {"pemilik": "xavv", "tanggal": "10-10-2026", "tidur": "23:00", "bangun":
"05:00", "kualitas": 4},
    2: {"pemilik": "xavv", "tanggal": "11-10-2026", "tidur": "02:30", "bangun":
"05:49", "kualitas": 3}
}


def id_baru():
    if len(data_tidur) == 0:
        return 1
    return max(data_tidur) + 1

File tampilan.py
import os
from prettytable import PrettyTable
from colorama import Fore, init
from data import data_tidur, akun

init(autoreset=True)


def bersihkan_layar():
    os.system("cls" if os.name == "nt" else "clear")


def sukses(teks):
    print(Fore.GREEN + teks)


def gagal(teks):
    print(Fore.RED + teks)


def jeda():
    input("Tekan Enter untuk lanjut.")


def hitung_durasi(d):
    mulai = int(d["tidur"][:2]) * 60 + int(d["tidur"][3:])
    akhir = int(d["bangun"][:2]) * 60 + int(d["bangun"][3:])
    durasi = akhir - mulai
```

```
    if durasi <= 0:
        durasi = durasi + 1440
    return durasi


def format_durasi(menit):
    return str(menit // 60) + " jam " + str(menit % 60) + " menit"


def tampil_menu(judul, pilihan):
    tabel = PrettyTable()
    tabel.title = judul
    tabel.field_names = ["Kode", "Menu"]
    tabel.align["Menu"] = "l"

    for kode, teks in pilihan:
        tabel.add_row([kode, teks])

    print(tabel)


def tabel_riwayat(daftar_id):
    tabel = PrettyTable()
    tabel.field_names = ["No", "Pemilik", "Tanggal", "Jam Tidur",
                         "Jam Bangun", "Durasi", "Kualitas"]

    for no in range(len(daftar_id)):
        d = data_tidur[daftar_id[no]]
        tabel.add_row([
            no + 1,
            d["pemilik"],
            d["tanggal"],
            d["tidur"],
            d["bangun"],
            format_durasi(hitung_durasi(d)),
            str(d["kualitas"]) + "/5"
        ])

    print(tabel)


def tabel_rata(rata):
    tabel = PrettyTable()
    tabel.field_names = ["Keterangan", "Hasil"]
    tabel.add_row(["Rata-rata durasi tidur", format_durasi(rata)])
    print(tabel)


def tabel_akun():
    tabel = PrettyTable()
```

```
    tabel.field_names = ["No", "Username", "Role"]

    no = 1
    for a in akun:
        tabel.add_row([no, a, akun[a]["role"]])
        no = no + 1

    print(tabel)
```

File validasi.py

```
from tampilan import gagal


def validasi_jam(jam):
    try:
        if len(jam) != 5 or jam[2] != ":":
            raise ValueError("Format jam harus HH:MM")
        if not jam[:2].isdigit() or not jam[3:].isdigit():
            raise ValueError("Jam dan menit harus angka")
        if int(jam[:2]) > 23 or int(jam[3:]) > 59:
            raise ValueError("Jam atau menit di luar batas!")
    except ValueError:
        return False
    else:
        return True


def input_tanggal():
    tanggal = input("Tanggal (DD-MM-YYYY): ")

    try:
        if (len(tanggal) != 10 or tanggal[2] != "-"
            or tanggal[5] != "-"
            or not tanggal[:2].isdigit()
            or not tanggal[3:5].isdigit()
            or not tanggal[6:].isdigit()):
            raise ValueError("Format tanggal salah! Contoh: 20-09-2026")

        hari = int(tanggal[:2])
        bulan = int(tanggal[3:5])
        tahun = int(tanggal[6:])

        if bulan < 1 or bulan > 12:
            raise ValueError("Bulan tidak valid!")

        batas_hari = [31, 28, 31, 30, 31, 30,
                      31, 31, 30, 31, 30, 31]

        if tahun % 400 == 0 or (tahun % 4 == 0 and tahun % 100 != 0):
            batas_hari[1] = 29
```

```
        if hari < 1 or hari > batas_hari[bulan - 1]:
            raise ValueError("Tanggal tidak valid!")
    except ValueError as e:
        gagal(str(e))
        return None
    else:
        return tanggal


def input_kualitas():
    kualitas = input("Kualitas tidur (1-5): ")

    try:
        kualitas = int(kualitas)
        if kualitas < 1 or kualitas > 5:
            raise ValueError
    except ValueError:
        gagal("Kualitas harus angka 1 sampai 5.")
        return None
    else:
        return kualitas
```

File [autent.py]

```
import pwinput
from data import akun
from tampilan import sukses, gagal, jeda


def register():
    username = input("Username baru: ")
    password = pwinput.pwinput("Password baru: ")

    ada = username in akun

    try:
        if username == "" or " " in username or len(username) < 3:
            raise ValueError("Username minimal 3 karakter tanpa spasi.")
        if ada:
            raise ValueError("Username sudah digunakan!")
        if len(password) < 4:
            raise ValueError("Password minimal 4 karakter!")
    except ValueError as e:
        gagal(str(e))
    else:
        akun[username] = {"password": password, "role": "user"}
        sukses("Register berhasil! Silakan login kembali.")
    finally:
        jeda()
```

```
def login():
    username = input("Username: ")
    password = pwinput.pwinput("Password: ")

    for a in akun:
        if a == username and akun[a]["password"] == password:
            return a

    gagal("Username atau password salah!")
    jeda()
    return None
```

File crud.py

```
from data import data_tidur, id_baru
from validasi import validasi_jam, input_tanggal, input_kualitas
from tampilan import (tabel_riwayat, tabel_rata, sukses, gagal, jeda,
                      hitung_durasi)


def ambil_data(login, role):
    hasil = []
    for i in data_tidur:
        d = data_tidur[i]
        if role == "admin" or d["pemilik"] == login:
            hasil.append(i)
    return hasil


def jumlah_durasi(daftar, n):
    if n == 0:
        return 0
    return daftar[n - 1] + jumlah_durasi(daftar, n - 1)


# CREATE
def tambah_data(login):
    tanggal = input_tanggal()

    if tanggal is None:
        jeda()
        return

    waktu = []
    for label in ["tidur", "bangun"]:
        while True:
            jam = input("Jam " + label + " (HH:MM): ")

            if validasi_jam(jam):
```

```
                waktu.append(jam)
                break

            gagal("Format jam salah! Contoh: 23:30")

    kualitas = input_kualitas()

    if kualitas is not None:
        data_tidur[id_baru()] = {
            "pemilik": login,
            "tanggal": tanggal,
            "tidur": waktu[0],
            "bangun": waktu[1],
            "kualitas": kualitas
        }
        sukses("Data berhasil ditambahkan!")

    jeda()


# READ
def lihat_data(login, role):
    tampil = ambil_data(login, role)

    if len(tampil) == 0:
        gagal("Belum ada data tidur!")
    else:
        tabel_riwayat(tampil)

        daftar_durasi = []
        for i in tampil:
            daftar_durasi.append(hitung_durasi(data_tidur[i]))

        total = jumlah_durasi(daftar_durasi, len(daftar_durasi))
        rata = total // len(daftar_durasi)
        tabel_rata(rata)

    jeda()


# UPDATE
def ubah_data(login, role):
    tampil = ambil_data(login, role)

    if len(tampil) == 0:
        gagal("Belum ada data tidur.")
        jeda()
        return

    tabel_riwayat(tampil)
    pilih_data = input("Nomor data yang diubah: ")
```

```
    try:
        nomor = int(pilih_data)
        if nomor < 1 or nomor > len(tampil):
            raise ValueError
    except ValueError:
        gagal("Nomor tidak valid!")
        jeda()
        return

    d = data_tidur[tampil[nomor - 1]]

    tidur = input("Tambahkan jam tidur baru (Tekan Enter jika tetap): ")
    bangun = input("Tambahkan jam bangun baru (Tekan Enter jika tetap): ")
    kualitas = input("Kualitas baru (Tekan Enter jika tetap): ")

    salah = False

    if tidur != "" and not validasi_jam(tidur):
        gagal("Jam tidur tidak valid!")
        salah = True

    if bangun != "" and not validasi_jam(bangun):
        gagal("Jam bangun tidak valid!")
        salah = True

    if kualitas != "":
        try:
            kualitas_baru = int(kualitas)
            if kualitas_baru < 1 or kualitas_baru > 5:
                raise ValueError
        except ValueError:
            gagal("Kualitas harus angka 1 sampai 5.")
            salah = True

    if salah:
        gagal("Ada input yang tidak valid, data tidak diubah.")
    else:
        if tidur != "":
            d["tidur"] = tidur
        if bangun != "":
            d["bangun"] = bangun
        if kualitas != "":
            d["kualitas"] = int(kualitas)
        sukses("Data berhasil diubah!")

    jeda()


# DELETE
def hapus_data(login, role):
```

```
    tampil = ambil_data(login, role)

    if len(tampil) == 0:
        gagal("Belum ada data tidur!")
        jeda()
        return

    tabel_riwayat(tampil)
    pilih_data = input("Nomor data yang dihapus: ")

    try:
        nomor = int(pilih_data)
        if nomor < 1 or nomor > len(tampil):
            raise ValueError
    except ValueError:
        gagal("Nomor tidak valid!")
    else:
        yakin = input("Yakin mau hapus data? (y/n): ")

        if yakin.lower() == "y":
            data_tidur.pop(tampil[nomor - 1])
            sukses("Data berhasil dihapus!")
        else:
            print("Penghapusan dibatalkan.")

    jeda()
```

File menu.py

```
import autent
import crud
from data import akun
from tampilan import bersihkan_layar, tampil_menu, tabel_akun, gagal, jeda


def menu_user(login, role):
    while True:
        bersihkan_layar()

        pilihan = [
            ("1", "Tambah data tidur"),
            ("2", "Lihat riwayat tidur"),
            ("3", "Ubah data tidur"),
            ("4", "Hapus data tidur")
        ]

        if role == "admin":
            pilihan.append(("5", "Lihat daftar akun"))

        pilihan.append(("0", "Logout"))
```

```
        tampil_menu("MENU " + login + " - " + role, pilihan)
        menu = input("Pilih menu: ")

        if menu == "1":
            crud.tambah_data(login)
        elif menu == "2":
            crud.lihat_data(login, role)
        elif menu == "3":
            crud.ubah_data(login, role)
        elif menu == "4":
            crud.hapus_data(login, role)
        elif menu == "5" and role == "admin":
            tabel_akun()
            jeda()
        elif menu == "0":
            break
        else:
            gagal("Menu tidak valid!")
            jeda()


def menu_utama():
    while True:
        bersihkan_layar()
        tampil_menu("PENCATAT WAKTU TIDUR", [
            ("1", "Login"),
            ("2", "Register"),
            ("3", "Keluar")
        ])
        pilih = input("Pilih menu: ")

        if pilih == "3":
            print("Program selesai.")
            break
        elif pilih == "2":
            autent.register()
        elif pilih == "1":
            login = autent.login()
            if login is not None:
                menu_user(login, akun[login]["role"])
        else:
            gagal("Pilihan tidak valid!")
            jeda()
```

File [main.py]

```
from menu import menu_utama

if __name__ == "__main__":
    menu_utama()
```

A. Fitur Tambah Data

Fitur ini digunakan pengguna untuk menambahkan data tidur baru dan berada di file crud.py pada fungsi tambah_data(). Program meminta tanggal, jam tidur, jam bangun, dan kualitas tidur, lalu memvalidasi setiap input melalui fungsi di validasi.py. Jika semua valid, data disimpan ke dictionary data_tidur dengan ID dari fungsi id_baru() sebagai key, sehingga ID tidak pernah sama.

```
def tambah_data(login):
    tanggal = input_tanggal()

    if tanggal is None:
        jeda()
        return

    waktu = []
    for label in ["tidur", "bangun"]:
        while True:
            jam = input("Jam " + label + " (HH:MM): ")

            if validasi_jam(jam):
                waktu.append(jam)
                break

            gagal("Format jam salah! Contoh: 23:30")

    kualitas = input_kualitas()

    if kualitas is not None:
        data_tidur[id_baru()] = {
            "pemilik": login,
            "tanggal": tanggal,
            "tidur": waktu[0],
            "bangun": waktu[1],
            "kualitas": kualitas
        }
        sukses("Data berhasil ditambahkan!")

    jeda()
```

B. Fitur Lihat Riwayat (Read)

Fitur ini berada di crud.py pada fungsi lihat_data(). Admin melihat data semua pengguna, sedangkan user hanya melihat datanya sendiri, yang dipilih lewat fungsi ambil_data(). Riwayat

ditampilkan dengan tabel PrettyTable. Durasi tiap data dihitung dalam menit oleh hitung_durasi() di tampilan.py dari selisih jam bangun dan jam tidur. Jika hasilnya 0 atau negatif (tidur melewati tengah malam), ditambah 1440 menit. Rata-rata durasi dihitung dengan fungsi rekursif jumlah_durasi().

```
def lihat_data(login, role):
    tampil = ambil_data(login, role)

    if len(tampil) == 0:
        gagal("Belum ada data tidur!")
    else:
        tabel_riwayat(tampil)

        daftar_durasi = []
        for i in tampil:
            daftar_durasi.append(hitung_durasi(data_tidur[i]))

        total = jumlah_durasi(daftar_durasi, len(daftar_durasi))
        rata = total // len(daftar_durasi)
        tabel_rata(rata)

    jeda()
```

C. Fitur Ubah Data (Update)

Fitur ini berada di crud.py pada fungsi ubah_data(). Program menampilkan tabel data yang boleh diakses (disimpan dalam list tampil berisi ID), lalu pengguna memilih nomor data. Jika input dikosongkan, nilai lama tetap dipakai. Semua input divalidasi dulu: jam dengan validasi_jam() dan kualitas dengan try-except. Data baru diubah hanya jika semuanya valid.

```
def ubah_data(login, role):
    tampil = ambil_data(login, role)

    if len(tampil) == 0:
        gagal("Belum ada data tidur.")
        jeda()
        return

    tabel_riwayat(tampil)
    pilih_data = input("Nomor data yang diubah: ")

    try:
        nomor = int(pilih_data)
```

```
        if nomor < 1 or nomor > len(tampil):
            raise ValueError
    except ValueError:
        gagal("Nomor tidak valid!")
        jeda()
        return

    d = data_tidur[tampil[nomor - 1]]

    tidur = input("Tambahkan jam tidur baru (Tekan Enter jika tetap): ")
    bangun = input("Tambahkan jam bangun baru (Tekan Enter jika tetap): ")
    kualitas = input("Kualitas baru (Tekan Enter jika tetap): ")

    salah = False

    if tidur != "" and not validasi_jam(tidur):
        gagal("Jam tidur tidak valid!")
        salah = True

    if bangun != "" and not validasi_jam(bangun):
        gagal("Jam bangun tidak valid!")
        salah = True

    if kualitas != "":
        try:
            kualitas_baru = int(kualitas)
            if kualitas_baru < 1 or kualitas_baru > 5:
                raise ValueError
        except ValueError:
            gagal("Kualitas harus angka 1 sampai 5.")
            salah = True

    if salah:
        gagal("Ada input yang tidak valid, data tidak diubah.")
    else:
        if tidur != "":
            d["tidur"] = tidur
        if bangun != "":
            d["bangun"] = bangun
        if kualitas != "":
            d["kualitas"] = int(kualitas)
        sukses("Data berhasil diubah!")

    jeda()
```

D. Fitur Hapus Data (Delete)

Fitur ini berada di crud.py pada fungsi hapus_data(). Program menampilkan tabel data, pengguna memilih nomor (divalidasi dengan try-except), lalu program meminta konfirmasi “y” atau “n”. Jika “y”, data dihapus dari dictionary dengan pop berdasarkan ID-nya.

```
def hapus_data(login, role):
    tampil = ambil_data(login, role)

    if len(tampil) == 0:
        gagal("Belum ada data tidur!")
        jeda()
        return

    tabel_riwayat(tampil)
    pilih_data = input("Nomor data yang dihapus: ")

    try:
        nomor = int(pilih_data)
        if nomor < 1 or nomor > len(tampil):
            raise ValueError
    except ValueError:
        gagal("Nomor tidak valid!")
    else:
        yakin = input("Yakin mau hapus data? (y/n): ")

        if yakin.lower() == "y":
            data_tidur.pop(tampil[nomor - 1])
            sukses("Data berhasil dihapus!")
        else:
            print("Penghapusan dibatalkan.")

    jeda()
```

E. Fitur Register dan Login

Fitur ini berada di autent.py. Pada register(), program memeriksa apakah username sudah ada, lalu memvalidasi di blok try: username minimal 3 karakter tanpa spasi dan belum dipakai, serta password minimal 4 karakter. Jika ada yang salah, program melakukan raise ValueError dan pesannya ditampilkan oleh except. Jika benar, akun baru disimpan dengan role “user” di blok else, dan blok finally selalu menampilkan “Tekan Enter untuk lanjut.” Password diinput dengan pwinput sehingga tampil sebagai bintang. Pada login(), program mencocokkan

username dan password, lalu mengembalikan username jika cocok atau None jika tidak. Role kemudian diambil dari akun[login][“role”] di menu.py.

```
def register():
    username = input("Username baru: ")
    password = pwinput.pwinput("Password baru: ")

    ada = username in akun

    try:
        if username == "" or " " in username or len(username) < 3:
            raise ValueError("Username minimal 3 karakter tanpa spasi.")
        if ada:
            raise ValueError("Username sudah digunakan!")
        if len(password) < 4:
            raise ValueError("Password minimal 4 karakter!")
    except ValueError as e:
        gagal(str(e))
    else:
        akun[username] = {"password": password, "role": "user"}
        sukses("Register berhasil! Silakan login kembali.")
    finally:
        jeda()


def login():
    username = input("Username: ")
    password = pwinput.pwinput("Password: ")

    for a in akun:
        if a == username and akun[a]["password"] == password:
            return a

    gagal("Username atau password salah!")
    jeda()
    return None
```

F. Fitur Admin

Menu nomor 5 hanya tampil dan bisa diakses oleh admin. Pengecekannya ada di menu.py, dan tabelnya dibuat oleh fungsi tabel_akun() di tampilan.py. Fitur ini menampilkan daftar username dan role semua akun dalam bentuk tabel PrettyTable.

```
elif menu == "5" and role == "admin":
    tabel_akun()
```

```
    jeda()

def tabel_akun():
    tabel = PrettyTable()
    tabel.field_names = ["No", "Username", "Role"]

    no = 1
    for a in akun:
        tabel.add_row([no, a, akun[a]["role"]])
        no = no + 1

    print(tabel)
```

G. Library Tambahan

Selain PrettyTable, program memakai dua library eksternal yang dipasang dengan pip. Library pwinput dipakai di autent.py agar password yang diketik tampil sebagai tanda bintang. Library colorama dipakai di tampilan.py lewat fungsi sukses() dan gagal(), sehingga pesan berhasil tampil hijau dan pesan gagal tampil merah. Fungsi init(autoreset=True) membuat warna kembali normal setelah setiap print.

```
password = pwinput.pwinput("Password: ")

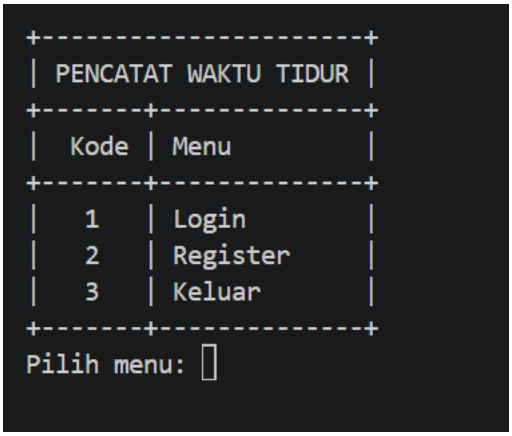
from colorama import Fore, init

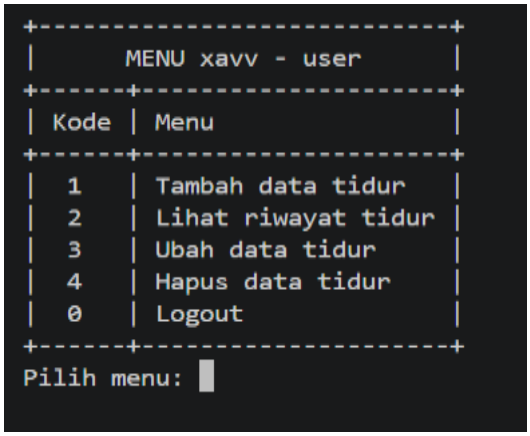
init(autoreset=True)

def sukses(teks):
    print(Fore.GREEN + teks)

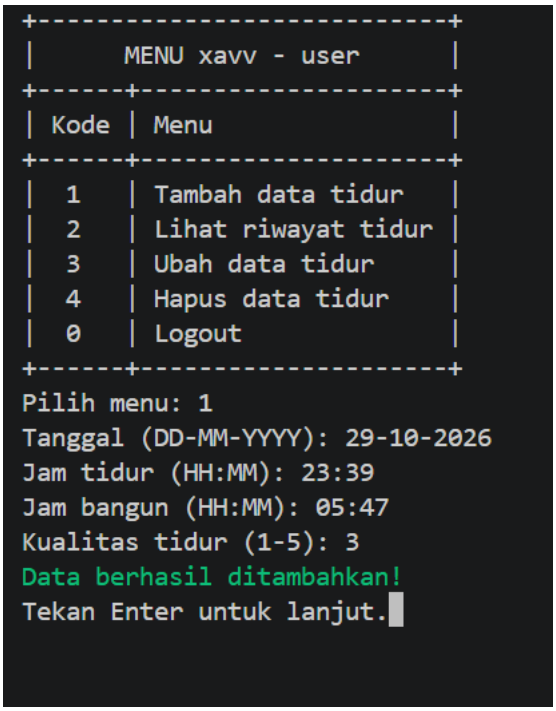
def gagal(teks):
    print(Fore.RED + teks)
```

4. Hasil Output

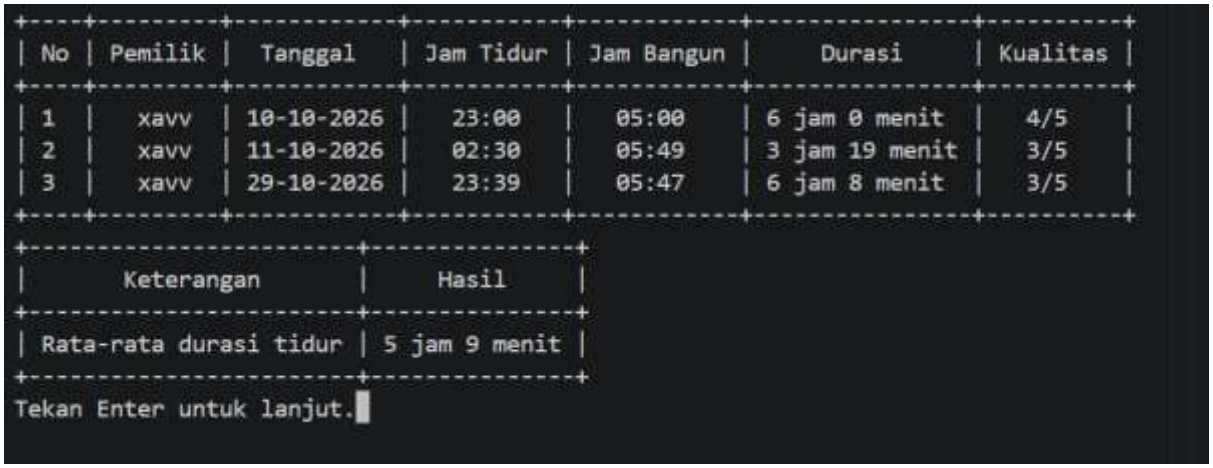




Gambar 4.1 dan 4.2 Tampilan Menu utama dan login berhasil



Gambar 4.3 Tampilan data berhasil di tambahkan

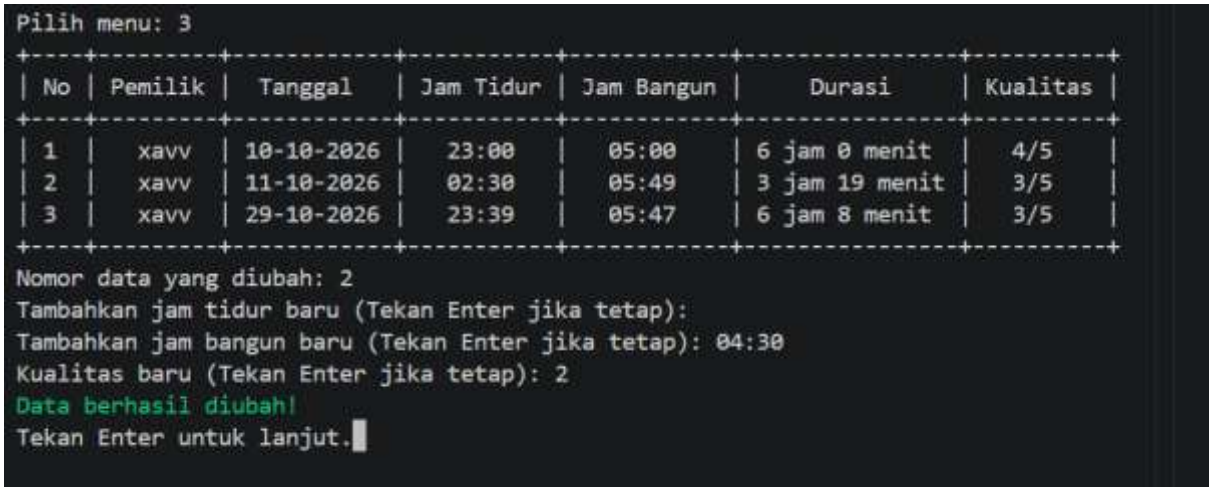


No	Pemilik	Tanggal	Jam Tidur	Jam Bangun	Durasi	Kualitas
1	xavv	10-10-2026	23:00	05:00	6 jam 0 menit	4/5
2	xavv	11-10-2026	02:30	05:49	3 jam 19 menit	3/5
3	xavv	29-10-2026	23:39	05:47	6 jam 8 menit	3/5

Keterangan	Hasil
Rata-rata durasi tidur	5 jam 9 menit

Tekan Enter untuk lanjut.

Gambar 4.4 Melihat riwayat tidur



Pilih menu: 3

No	Pemilik	Tanggal	Jam Tidur	Jam Bangun	Durasi	Kualitas
1	xavv	10-10-2026	23:00	05:00	6 jam 0 menit	4/5
2	xavv	11-10-2026	02:30	05:49	3 jam 19 menit	3/5
3	xavv	29-10-2026	23:39	05:47	6 jam 8 menit	3/5

Nomor data yang diubah: 2
Tambahkan jam tidur baru (Tekan Enter jika tetap):
Tambahkan jam bangun baru (Tekan Enter jika tetap): 04:30
Kualitas baru (Tekan Enter jika tetap): 2
Data berhasil diubah!
Tekan Enter untuk lanjut.

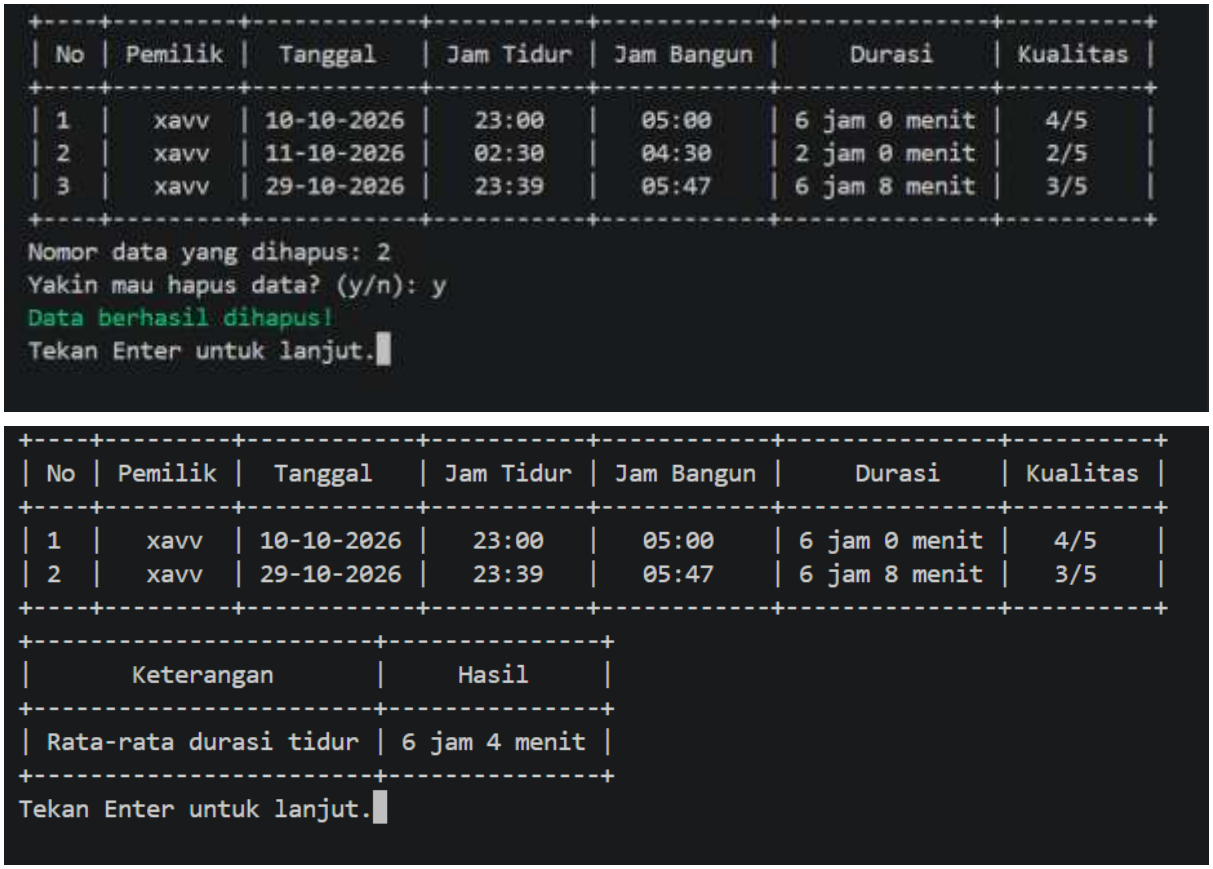


No	Pemilik	Tanggal	Jam Tidur	Jam Bangun	Durasi	Kualitas
1	xavv	10-10-2026	23:00	05:00	6 jam 0 menit	4/5
2	xavv	11-10-2026	02:30	04:30	2 jam 0 menit	2/5
3	xavv	29-10-2026	23:39	05:47	6 jam 8 menit	3/5

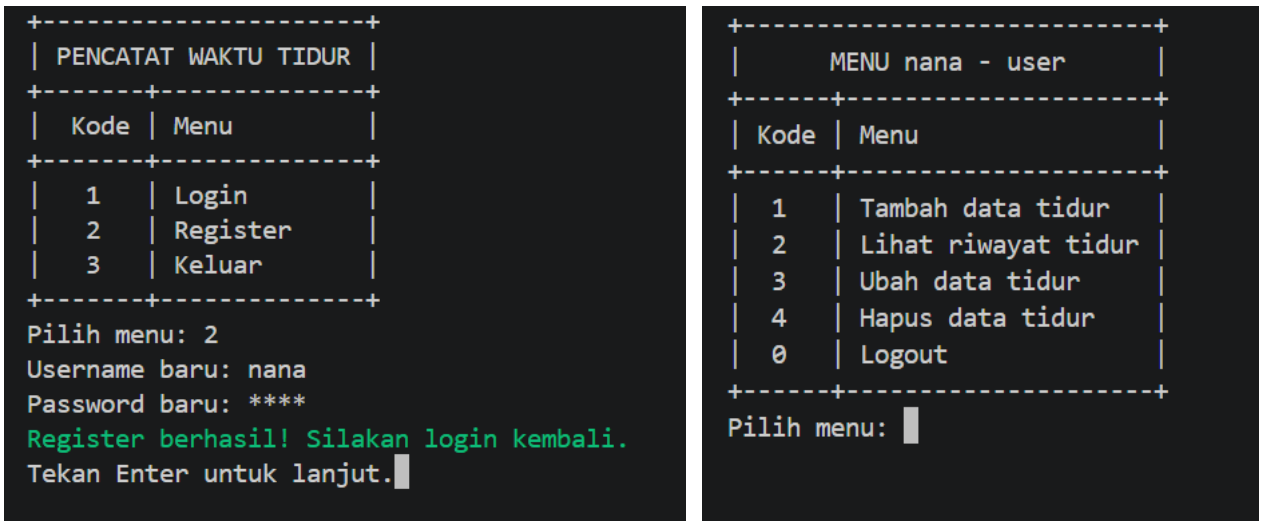
Keterangan	Hasil
Rata-rata durasi tidur	4 jam 42 menit

Tekan Enter untuk lanjut.

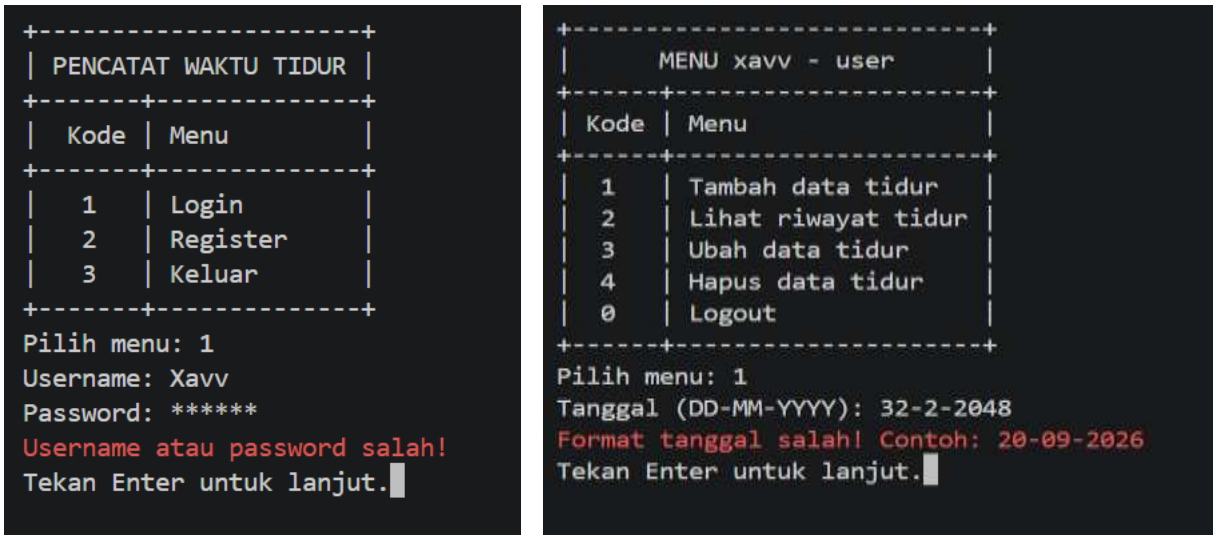
Gambar 4.5 dan 4.6 Mengubah data tidur dan tampilannya



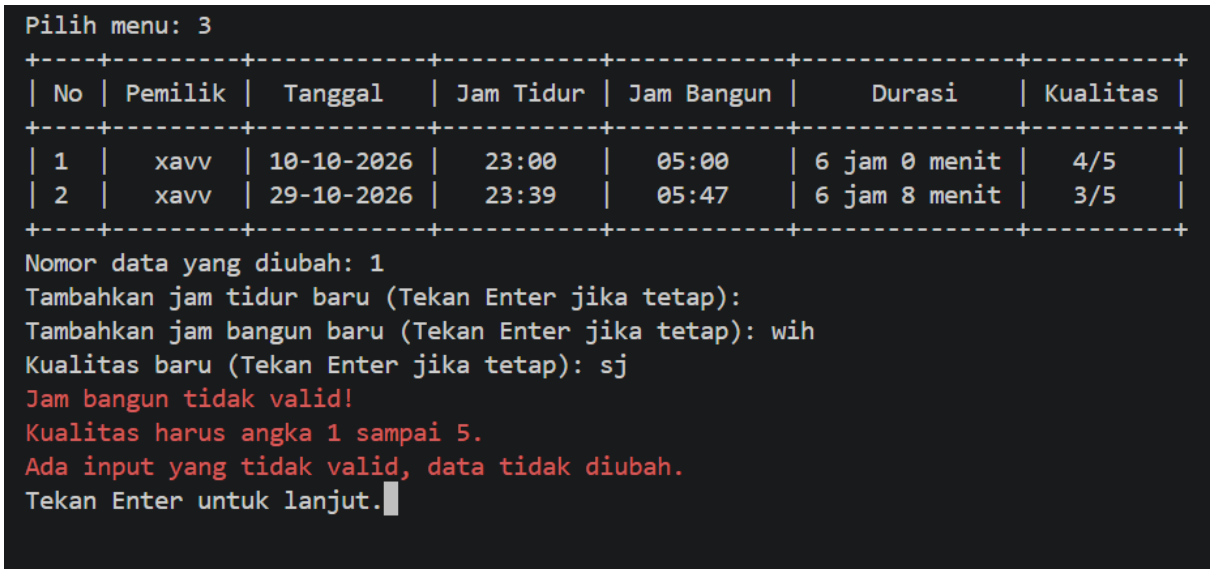
Gambar 4.7 dan 4.8 Menghapus data tidur dan tampilannya



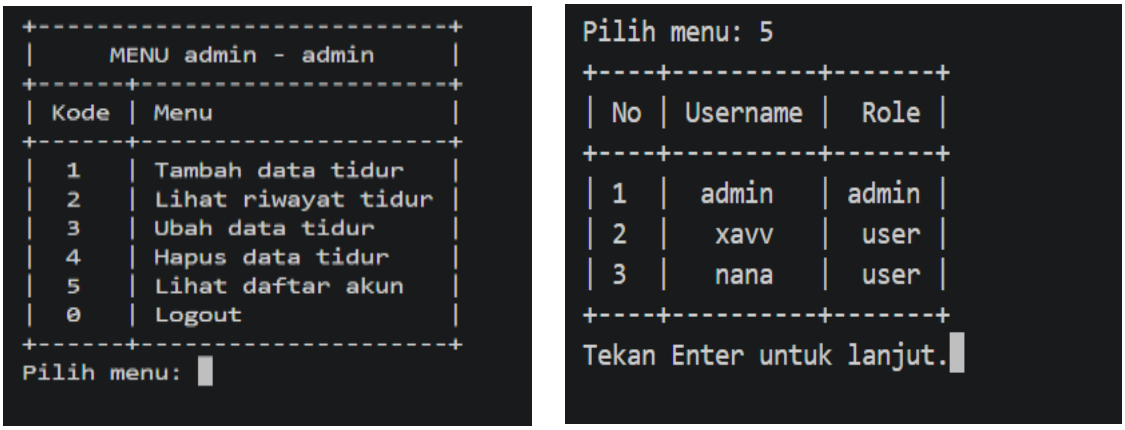
Gambar 4.9 dan 4.10 Register berhasil dan tampilannya



Gambar 4.11 dan 4.12 Username atau password salah dan Format tanggal salah



Gambar 4.13 Terdapat kesalahan saat mengimput jam bangun atau kualitas



Gambar 4.14 dan 4.15 berhasil login akun admin dan melihat daftar akun

5. Langkah-langkah Git

5. 1 Git Add

```
PS C:\Users\USER\OneDrive\Praktikum-APD-2026\POSTTEST> git add .
```

Gambar 5.1 Git Add

Git Add digunakan untuk menambahkan perubahan pada file, seperti file baru, file yang di modifikasi, atau file yang dihapus ke dalam staging area.

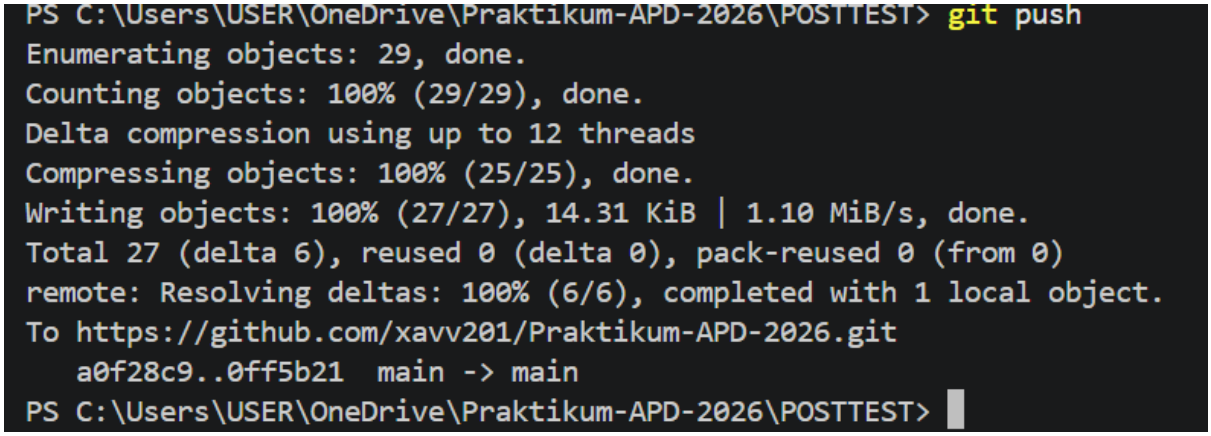
5.2 Git Commit

```
PS C:\Users\USER\OneDrive\Praktikum-APD-2026\POSTTEST> git commit -m "menambah file"
[main 68d82a8] menambah file
 13 files changed, 435 insertions(+)
 create mode 100644 POSTTEST/posttest-apd-8/posttest-apd-8/__pycache__/autent.cpython-313.pyc
 create mode 100644 POSTTEST/posttest-apd-8/posttest-apd-8/__pycache__/crud.cpython-313.pyc
 create mode 100644 POSTTEST/posttest-apd-8/posttest-apd-8/__pycache__/data.cpython-313.pyc
 create mode 100644 POSTTEST/posttest-apd-8/posttest-apd-8/__pycache__/menu.cpython-313.pyc
 create mode 100644 POSTTEST/posttest-apd-8/posttest-apd-8/__pycache__/tampilan.cpython-313.pyc
 create mode 100644 POSTTEST/posttest-apd-8/posttest-apd-8/__pycache__/validasi.cpython-313.pyc
 create mode 100644 POSTTEST/posttest-apd-8/posttest-apd-8/autent.py
 create mode 100644 POSTTEST/posttest-apd-8/posttest-apd-8/crud.py
 create mode 100644 POSTTEST/posttest-apd-8/posttest-apd-8/data.py
 create mode 100644 POSTTEST/posttest-apd-8/posttest-apd-8/main.py
 create mode 100644 POSTTEST/posttest-apd-8/posttest-apd-8/menu.py
 create mode 100644 POSTTEST/posttest-apd-8/posttest-apd-8/tampilan.py
 create mode 100644 POSTTEST/posttest-apd-8/posttest-apd-8/validasi.py
PS C:\Users\USER\OneDrive\Praktikum-APD-2026\POSTTEST>
```

Gambar 5.2 Git Commit

Git Commit digunakan untuk menyimpan perubahan yang sebelumnya telah dimasukkan ke staging area ke dalam riwayat repository lokal.

5.3 Git Push



Gambar 5.3 Git Push

Git Push digunakan untuk mengirim perubahan yang telah di-commit dari repository lokal ke repository remote/repository jarak jauh.